External Visual Memory Reranking for Frozen Game World Models

Anonymous CVPR submission

Paper ID AP0006

Abstract

Long-horizon world generation often drifts: objects disappear, spatial cues move, and the model forgets first-visit evidence needed for later consistency. This paper studies memory modules for game world models through the AP0006 Stage A/Stage C structure. Stage A is reserved for a teammate's forthcoming analysis of MatrixGame 3.0. Our implemented contribution is Stage C, a lightweight external memory module built on top of frozen MatrixGame-2 at inference time. The module writes first-visit visual evidence as object crops and descriptors, reads memory through patch-level feature matching between stored crops and sampled candidate frames, and reranks generated candidate videos. With approved road-sign memory, the selected output changes from seed1 to seed8. The resulting mechanism is lightweight, interpretable, and does not modify model weights, attention, or training.

1. Introduction

World models can generate visually plausible short clips, yet long-horizon consistency remains difficult. A scene may initially contain a useful landmark or target object, but later generations can forget where it was, replace it with distractors, or select a rollout that no longer respects earlier evidence. This motivates a practical memory question for generative world models: what should be stored, when should memory be read, and how should it influence later outputs?

Our AP0006 project follows a two-stage structure. Stage A is reserved for teammate analysis of MatrixGame 3.0 memory behavior. Our main implemented contribution is Stage C on frozen MatrixGame-2, where we add an inference-time external memory module without modifying the base generator. The module stores first-visit visual evidence, compares it against regular patches sampled from candidate videos, and reranks the candidate pool using an interpretable similarity score. The goal is not to improve raw realism, but to study whether a lightweight memory interface can change which generated rollout is selected for better object-level consistency.

2. Stage A: MatrixGame 3.0 Placeholder

TODO_TEAMMATE_MATRIXGAME3_STAGE_A

Team placeholder. This section is reserved for the teammate responsible for analyzing MatrixGame 3.0. The final version should identify where MatrixGame 3.0 writes, reads, and updates memory, what data structures represent memory, which code locations implement the mechanism, and how one concrete input example demonstrates memory influence. We intentionally leave this section compact in the current draft to avoid inventing unverified details.

3. Stage C: External Visual Memory Reranking

3.1. Write and store

Stage C targets frozen MatrixGame-2 with the generator kept unchanged throughout the pipeline. We implement memory entirely outside the model and apply it as external inference-time memory. When the first visit to a scene reveals a task-relevant object, such as a road sign, we write that evidence into a small memory bank as an object crop together with its feature vector and lightweight metadata. Typical fields include the crop path, frame index, candidate or scene identifier, and a short semantic tag indicating the target object. This design keeps the stored unit explicit and auditable: the memory bank contains visual evidence rather than hidden recurrent state. Figure 1 summarizes the full write-read-rerank pipeline.

3.2. Read, match, and score

At read time, the reranker evaluates a candidate set of generated videos indexed by j. From candidate j, we sample frames indexed by t and split each frame into regular grid patches indexed by k. Let m denote a stored memory crop, $p_{j,t,k}$ the k-th patch from frame t

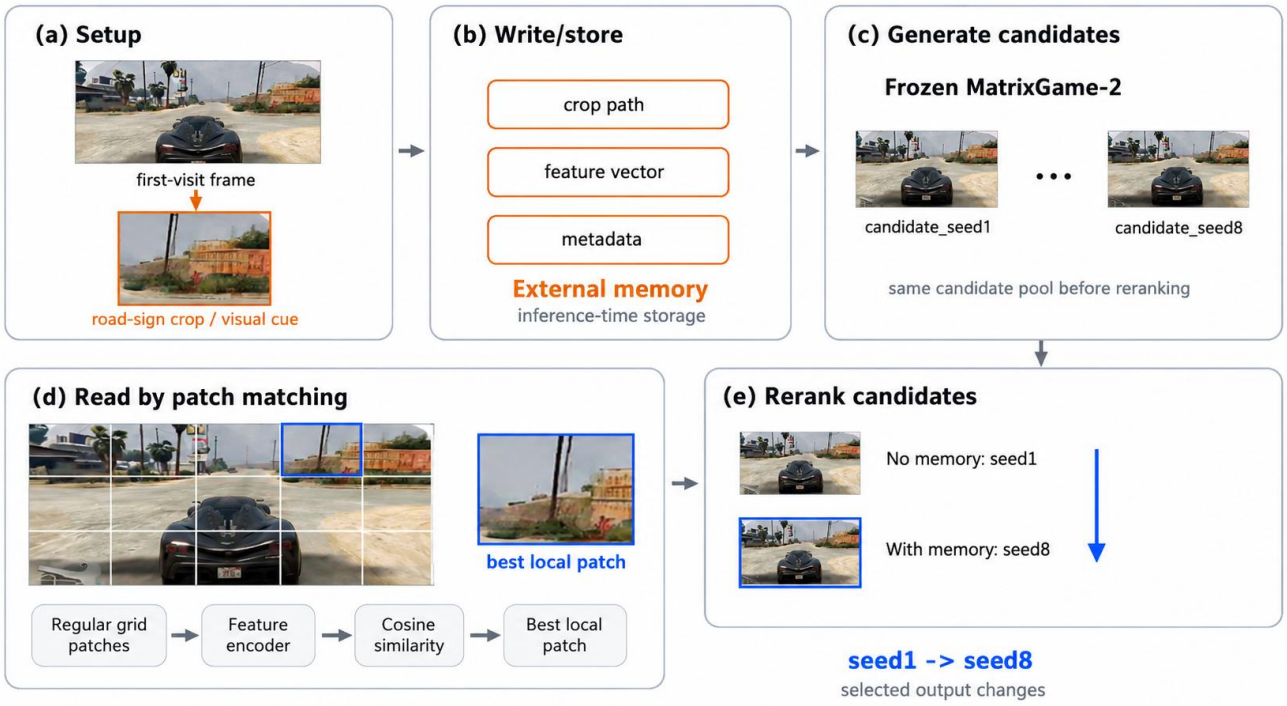

Figure 1. Overview of the external memory reranking pipeline. MatrixGame-2 is kept frozen. The memory module writes first-visit visual evidence, reads it through regular-grid patch matching, and reranks generated candidate videos. The main observed selection changes from seed1 without memory to seed8 with approved visual memory.

of candidate j, and $f(\cdot)$ a visual feature encoder. Depending on environment availability, f can be instantiated with DINOv2, CLIP, torchvision ResNet18, or a normalized RGB fallback. These components are used as feature extractors only; they are not segmentation models.

After feature normalization, we score each patch against memory using cosine-style similarity:

$$s_{j,t,k} = \cos\left(f(m), f\left(p_{j,t,k}\right)\right). \tag{1}$$

For each frame, we keep the strongest matching patches and average them:

$$S_{j,t} = \frac{1}{K'} \sum_{k \in \mathrm{TopK}} s_{j,t,k}. \tag{2}$$

Frame-level scores are then averaged across sampled frames to obtain a candidate-level memory score,

$$S_j = \frac{1}{T} \sum_t S_{j,t}, \qquad j^\star = \arg\max_j S_j. \tag{3}$$

The reranker finally selects the candidate with the largest S_j. Figure 2 illustrates this patch-level memory reading process with a stored crop, regular grid patches, and the selected best-matching cell. In words, memory is used neither to retrain the generator nor to alter its internal attention, but to select the rollout whose visible patches best agree with first-visit evidence.

Setting	Memory	Selected Seed	Interpretation
No memory	none	seed1	default choice
Road-sign memory	crop + patch match	seed8	selection switches

Table 1. External memory changes candidate selection in the MatrixGame-2 candidate pool.

4. Experiments and Results

Our current evidence is a compact case study centered on road-sign memory. The baseline system samples a pool of candidate videos from frozen MatrixGame-2 and selects a default candidate without external memory. The Stage C reranker reuses the same candidate pool but changes the final choice according to object-patch similarity. In the approved road-sign setting, the selected candidate changes from seed1 to seed8, showing that external memory can alter output selection even when the generator itself is unchanged.

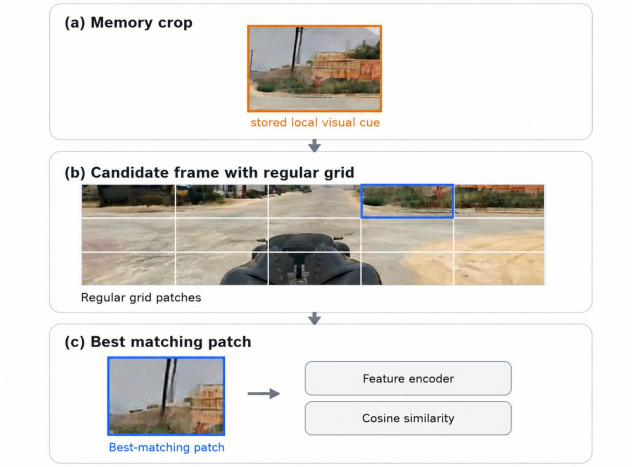

Figure 2. Patch-level memory read. A stored memory crop is compared with regular-grid patches from candidate frames using a visual feature encoder and cosine-style similarity. No object segmentation is used; patches are regular grid cells.

The accompanying project artifacts provide qualitative support for this switch: a generated comparison demo and the patch-level illustration in Figure 2. We treat these assets as evidence for candidate reranking, not as proof that MatrixGame-2 has acquired an internal memory mechanism. The contribution here is the external reranker itself: a simple memory card, a transparent matching rule, and an observable difference in which candidate is chosen.

5. Limitations and Conclusion

This study has several limitations. First, the mechanism is external reranking only; it does not modify model parameters, hidden state, or attention inside MatrixGame-2. Second, the current evidence is limited to a small candidate pool and a small number of scenarios, so the results should be interpreted as qualitative or small-scale case-study evidence rather than broad statistical validation. Third, Stage A remains pending teammate contribution, so the full paper will need a verified MatrixGame 3.0 analysis before final submission.

Even with these constraints, the project clarifies a useful design point for memory modules in world models. A lightweight bank of first-visit visual evidence, combined with patch-level matching and candidate reranking, can produce an interpretable change in selected output from seed1 to seed8 without retraining the base model. The contribution is therefore not raw generation quality, but an explicit memory mechanism for consistency-oriented candidate selection.

References